Health Transformation Review — Database Guide

Covers the Supabase PostgreSQL database schema, Row Level Security, and data access patterns.

Overview

HTR uses two data stores:

- **Supabase** (PostgreSQL) — authentication, user management, subscriptions, billing, and structured application data
- **Sanity CMS** — all editorial content (articles, modules, definitions, etc.)

Supabase also hosts the **pgvector** extension used by the AI RAG system for embedding storage.

Connection Details

- **Project URL**: `https://clryhwqaqhvdikgesjbc.supabase.co`
- **Region**: AWS (inferred from pooler URL)
- **Dashboard**: https://supabase.com/dashboard/project/clryhwqaqhvdikgesjbc

---

Client Patterns

Frontend — two clients

Anon client (`lib/db/client.ts` → `db`):

```
import { db } from "@/lib/db/client";

// Respects Row Level Security
// Safe in React Server Components and client components
const { data } = await db.from("profiles").select("full_name").eq("id",
userId).single();
```

Service role client (`lib/db/client.ts` → `dbAdmin`):

```
import { dbAdmin } from "@/lib/db/client";

// Bypasses Row Level Security
// Use ONLY in API routes and server actions
// NEVER import in client components
await dbAdmin.from("user_roles").upsert({ user_id, role });
```

Backend (Python)

```
from supabase import create_client
supabase = create_client(SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY)

# Query user roles
res = supabase.table("user_roles").select("role").eq("user_id",
user_id).execute()
roles = [r["role"] for r in (res.data or [])]
```

Database Tables

`profiles`

Stores user profile information. Automatically populated when a user signs up via Supabase Auth.

Column	Type	Description
`id`	`uuid` (PK)	Matches `auth.users.id`
`full_name`	`text`	Display name
`avatar_url`	`text`	Profile image URL

Access pattern: Anon client (`db`) with RLS — users can only read/write their own profile.

`user_roles`

Maps users to their access roles. A user can have multiple roles (e.g., `subscriber` and `student`). The system always uses the **highest** role.

Column	Type	Description
`user_id`	`uuid` (FK → `auth.users.id`)	The user
`role`	`text`	Role name

Valid roles (in ascending order): `free`, `subscriber`, `student`, `professional`, `advisory`, `admin`

Unique constraint: `(user_id, role)` — prevents duplicate role assignments.

Access pattern: Service role only for writes. The backend reads via service role to determine AI access.

How roles are granted:

1. New user signup → no row (treated as `free`)
2. Stripe `checkout.session.completed` → delete `free` row, insert paid role
3. Stripe `customer.subscription.deleted` → delete all paid roles, insert `free`
4. Manual admin grant → insert row directly via Supabase dashboard

---

`subscriptions`

Tracks Stripe subscription state per user. One row per user.

Column	Type	Description
`user_id`	`uuid` (PK/FK)	The user
`stripe_customer_id`	`text`	Stripe customer ID
`stripe_subscription_id`	`text`	Stripe subscription ID
`plan`	`text`	Plan name: `subscriber`, `student`, `professional`, `free`
`status`	`text`	Stripe status: `active`, `trialing`, `past_due`, `canceled`
`current_period_start`	`timestamptz`	Subscription period start
`current_period_end`	`timestamptz`	Subscription period end (renewal date)
`cancel_at_period_end`	`boolean`	Whether subscription cancels at period end

Upsert key: `user_id` — one subscription record per user, updated on each Stripe event.

Status flow:

```
(new user) → no row
  → checkout.session.completed → status: "active"
  → invoice.payment_failed     → status: "past_due"
  → subscription.updated       → status: "active"
  → subscription.deleted       → status: "canceled", plan: "free"
```

`stripe_customers`

Maps Supabase user IDs to Stripe customer IDs.

Column	Type	Description
`user_id`	`uuid` (FK)	Supabase user
`stripe_customer_id`	`text` (unique)	Stripe customer ID (e.g., `cus_...`)

Created during first checkout. Reused for subsequent checkouts to avoid duplicate Stripe customers.

`stripe_events`

Idempotency log for Stripe webhook events. Prevents duplicate processing if Stripe retries an event.

Column	Type	Description
`event_id`	`text` (PK)	Stripe event ID (e.g., `evt_...`)
`event_type`	`text`	Event type (e.g., `checkout.session.completed`)
`payload`	`jsonb`	Full Stripe event object

On webhook receipt: check if `event_id` exists → if yes, return 200 without processing.

`rag_documents`

The pgvector embedding table used by LlamaIndex for the AI knowledge base.

Column	Type	Description
`id`	Auto	Row ID
`embedding`	`vector(1536)`	OpenAI text-embedding-3-small vector
`text`	`text`	The document chunk text
`metadata`	`jsonb`	`{ source, doc_id, title, pillar }`

Managed entirely by LlamaIndex — do not modify manually. The `PGVectorStore` handles inserts, updates, and similarity queries.

This table is populated by `POST /api/ingest` (backend) and queried on every AI chat request.

Enabling pgvector: In Supabase Dashboard → Database → Extensions → enable `vector`.

Auth Flow

Supabase Auth handles signup, login, password reset, and sessions.

Session management

Uses `@supabase/ssr` for cookie-based sessions in Next.js. Sessions are stored in HTTP-only cookies and refreshed automatically.

Server-side session retrieval (`lib/auth.ts`):

```
const supabase = await createSupabaseServerClient();
const { data: { user } } = await supabase.auth.getUser();
```

Auth routes

- `POST /auth/v1/signup` — handled by Supabase (not Next.js)
- `POST /auth/v1/token` — login
- `/app/auth/callback/route.ts` — handles redirect after email magic link or OAuth

`getUser()` — full user fetch

Fetches three things in parallel:

1. `profiles` — `full_name`, `avatar_url`
2. `user_roles` — all roles for the user
3. `subscriptions` — current plan

Returns `AuthUser`:

```
{
  id: string;
  email: string;
  fullName: string | null;
  avatarUrl: string | null;
  role: "free" | "subscriber" | "student" | "professional" | "advisory" |
"admin";
  plan: string;
}
```

Row Level Security (RLS)

The anon client (`db`) respects RLS policies. The service role client (`dbAdmin`) bypasses them.

Recommended RLS policies

`profiles`:

```
-- Users can read and update their own profile
CREATE POLICY "users_own_profile" ON profiles
  FOR ALL USING (auth.uid() = id);
```

`user_roles`:

```
-- Users can read their own roles
CREATE POLICY "users_read_own_roles" ON user_roles
  FOR SELECT USING (auth.uid() = user_id);
-- Writes go through service role only (no user policy for
INSERT/UPDATE/DELETE)
```

`subscriptions`:

```
-- Users can read their own subscription
CREATE POLICY "users_read_own_sub" ON subscriptions
```

```
    FOR SELECT USING (auth.uid() = user_id);
```

`rag_documents`:

```
-- Service role only (no public access)
-- LlamaIndex uses service role connection via SUPABASE_DB_URL
```

Supabase Data Modules (`lib/db/`)

The `lib/db/` directory contains organized query modules for structured data.

Module	Purpose
`client.ts`	Exports `db` (anon) and `dbAdmin` (service role)
`academy.ts`	Academy course and enrollment queries
`hospitals.ts`	Hospital financial and performance data
`states.ts`	State profile and dashboard queries
`rht-profiles.ts`	RHT program profile queries
`learning-tracks.ts`	Learning track enrollment and progress
`state-initiatives.ts`	State initiative tracking
`time-series.ts`	HTI time-series data for dashboard charts

Static/Seed Data (`lib/data/`)

Some data is stored as TypeScript files rather than in Supabase. This is used for:

- Initial seeding before Supabase tables are populated
- Data that changes infrequently and doesn't need a database query
- Reference data used in multiple places

File	Contents
`states.ts`	State list (name, slug, abbreviation)
`hospital-data.ts`	Hospital financial scorecard data
`hti-timeseries-data.ts`	HTI index historical data points
`learning-tracks-data.ts`	Learning track definitions
`performance-index-data.ts`	State performance index scores
`rht-awards.ts`	RHT program award amounts by state
`rht-program.ts`	RHT program descriptions
`state-initiatives-data.ts`	State initiative data

Seed Scripts

Located in `frontend/scripts/`. Run with Node.js. Require appropriate env variables.

Script	Purpose
`seed-supabase.js`	Populate Supabase tables with initial data
`seed-academy-content.js`	Seed Academy content to Sanity

Script	Purpose
seed-ticker.js	Seed ticker/vitals data to Sanity
seed-caseStudies.js	Seed case study documents to Sanity
seed-reports.js	Seed report documents to Sanity
seed-webinars.js	Seed webinar documents to Sanity
import-glossary.js	Import glossary definitions to Sanity
import_academy.js	Bulk import academy modules to Sanity
bulk_import.js	General bulk import to Sanity
reset-database.js	Reset Supabase tables (destructive)

Usage example:

```
cd frontend
node scripts/seed-supabase.js
```

Ensure `.env.local` is set before running any script.

Common Database Operations

Grant a user a role manually

```
-- Via Supabase SQL editor
INSERT INTO user_roles (user_id, role)
VALUES ('user-uuid-here', 'advisory')
ON CONFLICT (user_id, role) DO NOTHING;
```

Downgrade a user to free

```
DELETE FROM user_roles
WHERE user_id = 'user-uuid-here'
  AND role IN ('subscriber', 'student', 'professional', 'advisory');

INSERT INTO user_roles (user_id, role)
VALUES ('user-uuid-here', 'free')
ON CONFLICT (user_id, role) DO NOTHING;
```

Check a user's current role and subscription

```
SELECT
  u.email,
  ur.role,
  s.plan,
  s.status,
  s.current_period_end
FROM auth.users u
LEFT JOIN user_roles ur ON u.id = ur.user_id
LEFT JOIN subscriptions s ON u.id = s.user_id
WHERE u.email = 'user@example.com';
```

Find all active subscribers

```
SELECT u.email, s.plan, s.current_period_end
FROM subscriptions s
JOIN auth.users u ON s.user_id = u.id
WHERE s.status = 'active'
ORDER BY s.current_period_end;
```

Count documents in the RAG vector store

```
SELECT COUNT(*), metadata->>'source' as source
FROM rag_documents
GROUP BY source
ORDER BY count DESC;
```

Find past-due subscriptions

```
SELECT u.email, s.current_period_end
FROM subscriptions s
JOIN auth.users u ON s.user_id = u.id
WHERE s.status = 'past_due';
```

Backup and Recovery

Supabase provides:

- **Automatic daily backups** (retention depends on plan tier)
- **Point-in-time recovery** (PITR) on Pro plan and above
- **Manual backups** via Supabase Dashboard → Settings → Backups

The `rag_documents` table can be rebuilt at any time by running `POST /api/ingest` — it is not considered critical backup data since it's derived from Sanity and PDF content.

Environment Variables

Variable	Where Used	Description
`NEXT_PUBLIC_SUPABASE_URL`	Frontend (public)	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Frontend (public)	Row Level Security anon key
`SUPABASE_SERVICE_ROLE_KEY`	Frontend API routes (server-only)	Bypasses RLS — server use only
`SUPABASE_URL`	Backend Python	Same URL, different env var name
`SUPABASE_SERVICE_ROLE_KEY`	Backend Python	Same service role key
`SUPABASE_JWT_SECRET`	Backend Python	For JWT verification (Settings > API)
`SUPABASE_DB_URL`	Backend Python	Direct PostgreSQL URL for pgvector

The `SUPABASE_DB_URL` format:

```
postgresql://postgres.[project-ref]:[password]@aws-0-us-east-
1.pooler.supabase.com:6543/postgres
```

Get this from: Supabase Dashboard → Settings → Database → Connection Pooling → Transaction mode.